

Trilha	Fórum
video	03:34
Projeto 2 - Expandindo Arrays Aninhados	05:22
video	09:13
Projeto 2 - Loops e Repetições	06:14
video	07:14
Projeto 2 - Pivot e Unpivot	01:39
video	00:28
Projeto 2 - Split Functions	01:20
video	01:32
Projeto 2 - Método Collect no Spark	02:01
ebook	
Projeto 2 - Operadores	
ebook	
Projeto 2 - WithColumn	
ebook	
Projeto 2 - Operações de Colunas	
ebook	
Projeto 2 - MapType	
ebook	



WithColumn

No Apache Spark, o método **withColumn** é uma operação muito utilizada quando se trabalha com DataFrames.

Este método permite que você adicione uma nova coluna ao DataFrame ou substitua uma coluna existente com uma nova baseada em uma expressão ou valor existente.

withColumn é parte das transformações DataFrame API e é frequentemente usado para derivar novos dados, formatar dados existentes ou incorporar cálculos complexos em um DataFrame.

Sintaxe Básica

A sintaxe básica do **withColumn** é:

```
1 DataFrame.withColumn(colName, col)
2
```

Onde:

colName: String que representa o nome da nova coluna ou da coluna que será substituída.

col: Uma coluna que é uma expressão que pode ser calculada a partir de colunas existentes.

Exemplos de Uso

Aqui estão alguns exemplos demonstrando como o withColumn pode ser usado para manipular DataFrames em Spark:

Adicionar uma Nova Coluna

Suponhamos que temos um **DataFrame** **df** com colunas **id** e **value** e queremos adicionar uma nova coluna chamada **square** que contém o quadrado dos valores na coluna **value**.

```
1 from pyspark.sql import SparkSession
2 from pyspark.sql.functions import col
3
4 # Inicializa o SparkSession
5 spark = SparkSession.builder \
6     .appName("WithColumn Example") \
7     .getOrCreate()
8
9 # Dados de exemplo
10 data = [(1, 3), (2, 6), (3, -1), (4, 0)]
11 columns = ["id", "value"]
12
13 # Criar DataFrame
14 df = spark.createDataFrame(data, columns)
15
16 # Adicionar uma nova coluna 'square'
17 df = df.withColumn("square", col("value") ** 2)
18
19 df.show()
20
```

Modificar uma Coluna Existente

Se quiser modificar uma coluna existente, por exemplo, para transformar a coluna **value** em seu valor absoluto:

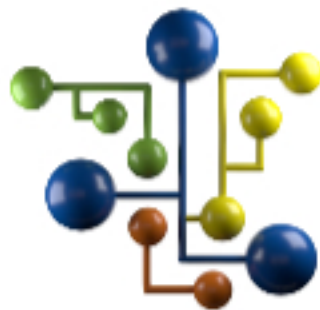
```
1 from pyspark.sql.functions import abs
2
3 # Modificar a coluna 'value'
4 df = df.withColumn("value", abs(col("value")))
5
6 df.show()
7
```

Utilizar Condições

Você também pode usar condições dentro de **withColumn** para criar colunas calculadas com base em condições lógicas. Por exemplo, adicionar uma coluna que classifica os valores:

```
1 from pyspark.sql.functions import when
2
3 df = df.withColumn("category", when(col("value") > 3, "High")
4
5 df.show()
6
```

O método **withColumn** é uma ferramenta poderosa na API DataFrame do Spark, permitindo uma ampla gama de manipulações de dados e transformações complexas de maneira eficiente e escalável.



Equipe DSA

Continue trilhando uma excelente
jornada de aprendizagem.